

Documentación proyecto I Estructura de Datos

Estudiante: Esteban Mena Cascante

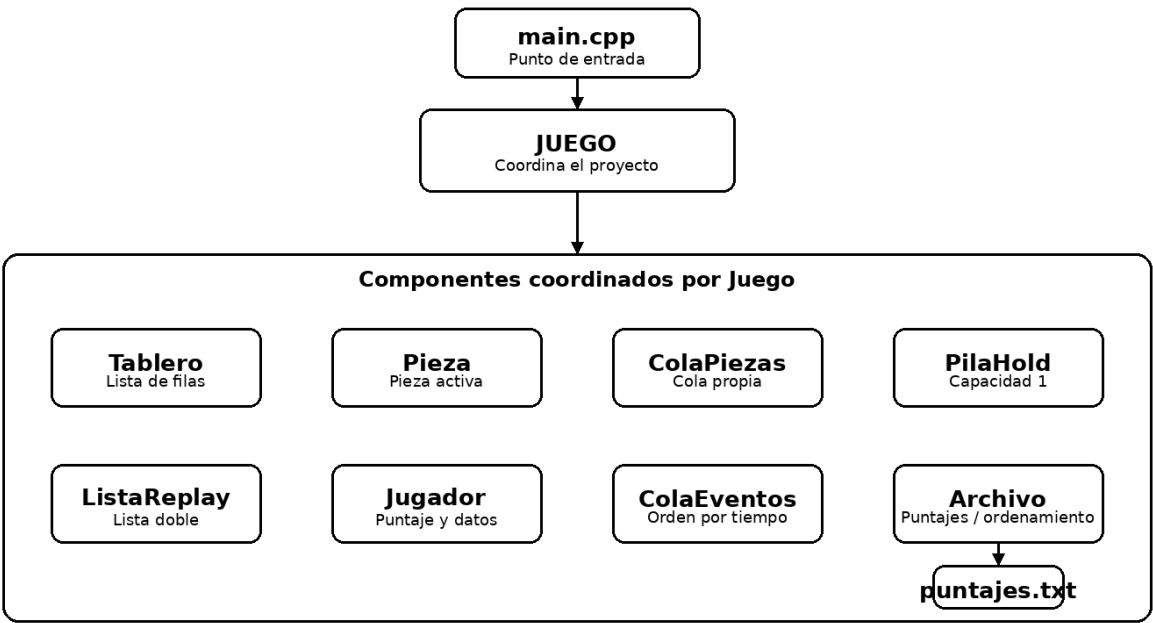
1. Estructuras utilizadas

El proyecto consiste en una versión de Tetris desarrollada en C++ y Raylib. Las mecánicas principales utilizan estructuras de datos propias para administrar las piezas, el Hold, el historial, los eventos y las filas del tablero.

Estructura	Uso principal
ColaPiezas	Cola propia para las próximas piezas.
PilaHold	Pila de capacidad uno para guardar una pieza.
ListaReplay	Lista doble para Undo, Redo y Replay.
ColaEventos	Cola enlazada ordenada por tiempo de activación.
Tablero	Lista enlazada de 20 filas, cada una con 10 celdas.
Archivo	Carga, guarda y ordena los mejores puntajes.

La clase Juego funciona como coordinador y contiene directamente el tablero, la pieza activa, la cola de piezas, el Hold, el historial, el jugador, el gestor de archivos y la cola de eventos.

Relación general de las estructuras



2. Complejidades Big-O

Las complejidades se determinaron según la forma en que están implementadas las operaciones dentro del proyecto.

Operación	Estructura	Complejidad
Encolar pieza	ColaPiezas	$O(1)$
Desencolar pieza	ColaPiezas	$O(1)$
Consultar próxima pieza	ColaPiezas	$O(n)$
Guardar / sacar Hold	PilaHold	$O(1)$
Registrar estado	ListaReplay + Tablero	$O(n^2)^*$
Undo / Redo (mover nodo)	ListaReplay	$O(1)^{**}$
Replay completo	ListaReplay + Tablero	$O(m \cdot n^2)^{***}$

Recorrer tablero	Tablero	$O(n)$
Eliminar fila	Tablero	$O(n)$
Insertar fila vacía	Tablero	$O(1)$
Insertar evento	ColaEventos	$O(n)$
Extraer próximo evento	ColaEventos	$O(1)$
Insertion Sort	Archivo	$O(n^2)$
MergeSort	Archivo	$O(n \log n)$

* Registrar un estado copia el tablero y obtener cada celda necesita localizar su fila en la lista.

** La navegación entre nodos es $O(1)$. La operación completa de Undo/Redo también restaura las celdas del tablero.

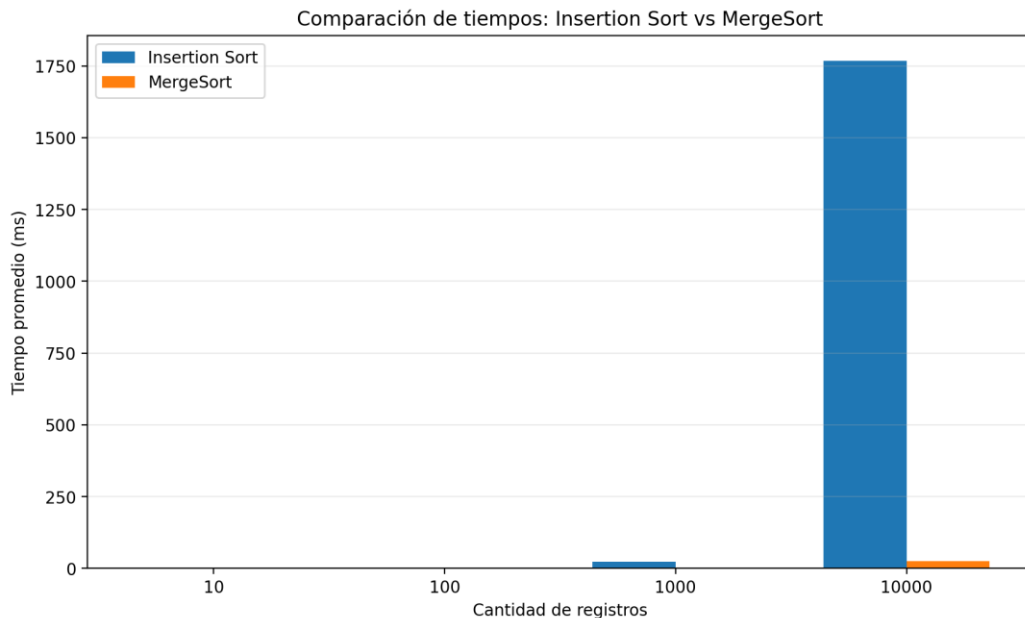
*** m es la cantidad de estados guardados y n la cantidad de filas. Como el tablero siempre es 20×10 , en la práctica el costo del Replay crece principalmente con la cantidad de estados.

La idea principal es que algunas estructuras hacen más trabajo al insertar o localizar un elemento, pero permiten que otras operaciones importantes sean directas. Por ejemplo, ColaEventos inserta en $O(n)$ para conservar el próximo evento al frente y extraerlo en $O(1)$.

3. Pruebas de Insertion Sort vs MergeSort

Para comparar los algoritmos propios se generaron puntajes temporales para 10, 100, 1 000 y 10 000 registros. En cada prueba se crearon dos copias idénticas de los datos, una para cada algoritmo. Cada tamaño se ejecutó cinco veces y se utilizó el promedio, medido en microsegundos.

Cantidad	Insertion Sort	MergeSort
10	9 μs	27 μs
100	244 μs	223 μs
1 000	22 598 μs	2 906 μs
10 000	1 767 993 μs	25 545 μs



Con cantidades pequeñas los tiempos son similares. A partir de 1 000 registros la diferencia se vuelve clara y, con 10 000, Insertion Sort aumenta mucho más que MergeSort.

4. Preguntas de análisis

1. Comparación entre $O(n^2)$ y $O(n \log n)$

Con 10 registros, Insertion Sort obtuvo 9 μs y MergeSort 27 μs , por lo que Insertion fue más rápido para esa cantidad pequeña. Con 100 registros los resultados fueron muy similares: 244 μs y 223 μs respectivamente.

A partir de aproximadamente 1 000 registros la diferencia se vuelve clara. Insertion Sort tardó 22 598 μs y MergeSort 2 906 μs . Con 10 000 registros la diferencia aumentó a 1 767 993 μs contra 25 545 μs .

Los resultados coinciden con lo esperado por la notación asintótica. Insertion Sort tiene complejidad $O(n^2)$, por lo que su tiempo aumenta rápidamente cuando crece la cantidad de datos. MergeSort tiene complejidad $O(n \log n)$, por lo que maneja mejor entradas grandes. Con pocos elementos, el trabajo adicional de dividir y combinar puede hacer que MergeSort no sea más rápido.

2. ¿Por qué representar el tablero con una lista enlazada de filas?

El tablero utiliza una lista enlazada donde cada nodo representa una fila de 10 celdas. Esta representación es razonable para la limpieza de líneas porque, cuando una fila está completa, se puede eliminar el nodo correspondiente y después insertar una nueva fila vacía al inicio.

En mi implementación, insertar una fila vacía al inicio tiene costo $O(1)$, porque se crea el nuevo nodo, se enlaza con el primero actual y se actualiza el puntero primero. Eliminar una fila tiene costo $O(n)$ en el peor caso, porque antes de eliminarla se debe recorrer la lista hasta encontrar el nodo anterior.

Una ventaja de este diseño es que no es necesario copiar manualmente el contenido de todas las filas superiores para simular su desplazamiento después de limpiar una línea.

3. ¿Por qué el replay requiere una lista doblemente enlazada y no una simplemente enlazada? ¿Qué costo tiene, en su implementación, retroceder o avanzar un paso?

Se utilizó una lista doblemente enlazada porque el Replay necesita moverse tanto al estado anterior como al siguiente. Cada nodo tiene un puntero anterior y un puntero siguiente.

Mover el puntero actual un paso hacia atrás o hacia adelante tiene costo $O(1)$, ya que el nodo vecino está disponible directamente. Con una lista simplemente enlazada sería sencillo avanzar, pero para retroceder habría que buscar nuevamente el nodo anterior desde el inicio.

En el juego, después de mover el nodo también se restaura el tablero guardado. Esa restauración tiene un costo adicional, pero la navegación propia de la lista continúa siendo $O(1)$.

4. ¿Como garantiza su implementación que la cola de eventos programados siempre mantenga al frente el evento más próximo a dispararse? ¿Qué complejidad tiene insertar un nuevo evento?

ColaEventos mantiene los nodos ordenados según su tiempo de activación. Cuando se inserta un evento nuevo, primero se comprueba si debe ir antes que el frente. Si no es así, se recorre la cola hasta encontrar la posición que le corresponde.

Por esta razón, insertar un evento tiene complejidad $O(n)$ en el peor caso. Ese trabajo durante la inserción permite que el evento con menor tiempo de activación permanezca siempre al frente, por lo que consultarlo o extraerlo se puede realizar en $O(1)$.

Cuando dos eventos tienen el mismo tiempo de activación, el nuevo se coloca después de los que ya estaban guardados con ese mismo tiempo, manteniendo su orden de llegada.